Technical Brief: Link-Health Supervisor for Aalyria Spacetime

Oct 24, 2026 · @alex mannino

Summary

A link-health supervisor for optical terminals that connects to Spacetime through the SDN agent's documented external-process hooks, with no changes to Aalyria's code.

It receives beam commands from Spacetime, watches the terminal's fine-steering mirror and lock signal, decides when the mirror should offload or re-lock, and reports link state back to Spacetime. Its design goal is that Spacetime only ever hears accurate information: a link is reported up only when light is actually arriving, and a schedule change is acknowledged only when it was actually carried out.

How it connects

The supervisor uses only the agent's published extension points: an external enactment command, an external telemetry command, and the standard agent configuration.

Spacetime interface	Supervisor component	Direction	Message
Scheduling API, via agent enactment hook	`qsd_enact.py`	Spacetime → terminal	`CreateEntryRequest` with `update_beam` / `delete_beam`
Telemetry API, via agent telemetry hook	`qsd_telemetry.py`	Terminal → Spacetime	`ExportMetricsRequest`: interface state, plus measured SINR / Es/N0 when available
Agent configuration	`agent_config.textproto`	Startup	`AgentParams` wiring both hooks
Terminal control loop	`qsd_daemon.py` + `qsd_stack_v2.py`	Local, at terminal rate	Mirror position, lock signal, offload and re-lock commands

The control loop runs as its own process at terminal speed; the hooks only exchange state files with it, so Spacetime's roughly 1 Hz telemetry cadence never holds up pointing control.

What was verified

Every message was checked against Python classes generated from Aalyria's own schema files at `github.com/aalyria/api` commit `ee3f0ea` (Oct 24, 2026), with unknown fields rejected.

Check	Result
Beam add, delete and retarget parsed from real `CreateEntryRequest` messages	Pass
Telemetry output parses as `telemetry.v1alpha.ExportMetricsRequest`	Pass
`agent_config.textproto` parses as `AgentParams`	Pass
Malformed request rejected with INVALID_ARGUMENT	Pass

Check	Result
Unhandled change type (e.g. `SetRoute`) returns UNIMPLEMENTED, not success	Pass
Rejected pointing command returns UNAVAILABLE; a retry re-sends it	Pass
End-to-end integration suite	17 of 17 pass
Tooling suite (record, replay, calibration, telemetry modes)	13 of 13 pass
Supervisor unit tests	17 of 17 pass
Repository verification script (`scripts/verify_qsd_spacetime.sh`)	Pass

The repository vendors an unmodified copy of `aalyria/api` at that commit, with Aalyria's Apache-2.0 LICENSE and NOTICE retained.

Link-health behavior

The supervisor is built so that what Spacetime is told matches what the terminal is actually doing.

- **Up means light is arriving.** Lock is taken from received light, not from the pointing-error estimate, which can read as centered when no light is present. While the supervisor is starting up or recovering, the link is reported dormant until it has been validated.
- **Recovers from glitches on its own.** A stale, missing or malformed sample puts the link in a temporary degraded state with no actuator commands; it returns to service after a run of good samples. In testing, a single 60 ms data dropout cost 0.5% of uptime instead of taking the link offline. Only bad data persisting for 5 seconds latches a fault.
- **Rides through brief blinks (optional).** A configurable coast time keeps a short loss of lock from triggering a full re-acquisition.
- **Keeps the mirror inside its range.** Offloads to coarse pointing remain active even while recovering, so a glitch cannot let the mirror drift to its limit.
- **Acknowledges only what it did.** Schedule changes it does not handle return UNIMPLEMENTED (or are handed to the node's existing enactment backend), and a pointing command the terminal rejects returns UNAVAILABLE.

- **Reports measured values only.** Modem metrics use provisioned modem IDs and measured SINR or Es/N0; nothing is estimated or invented.

Repository and how to run it

Repository: github.com/apmannino-design/qsd-phase-bounded-transport, branch `cursor/qsd-spacetime-upgrades-b2bf`, folder `qsd_spacetime/`.

```
pip install protobuf numpy googleapis-common-protos
cd qsd_spacetime
python3 test_integration.py        # end-to-end bridge test
python3 test_upgrades.py           # tooling
python3 qsd_stack_v2.py test       # supervisor unit tests
../scripts/verify_qsd_spacetime.sh # full verification against the vendored
API
```

Key settings: `QSD_MODEM_MAP` (provisioned modem IDs), `QSD_ENACT_CHAIN` (hand non-beam changes to the node's existing backend), `lock_loss_hold_s` (coast time), and `QSD_HW_*` hooks for the terminal driver.

Scope and open questions

All testing so far is in simulation, validated against Aalyria's published schemas; the next step is running inside the Go agent and on terminal hardware.

Not yet done:

- Running the hooks inside the compiled Go agent (Bazel build) against a live Spacetime instance
- Connecting the hardware hooks to a real terminal driver and calibrating thresholds from recorded data

Questions for the Spacetime team:

1. The repository's `ModemMetrics` (`demodulator_id`, `sinr_data_points`, `esn0_data_points`) differs from the online docs' legacy data model (`modem_id`, `link_metrics_data_points`). Which should a new integration target? The supervisor supports both.
2. How does Spacetime act on an interface reported as `IF_OPER_STATUS_DORMANT`?

3. What modem-ID convention should a terminal use so its metrics match the network model?